

■ RetailPulse

AI-Powered Customer Analytics & Demand Forecasting Platform

Predictive Demand · Customer Segmentation · Churn Analysis · Inventory Optimization

Advanced Data Science & Analytics Project
Zidio Development — Data Science & Analytics Domain
Portfolio / Interview Reference · March 2026 · Version 2.0

Prepared for: Zidio Development
Date: March 2026

1. Project Overview

Retailers routinely lose revenue to poor demand forecasting and stock mismanagement. RetailPulse is an end-to-end data science platform that ingests retail sales, customer, and inventory data to produce demand forecasts, customer segments, churn risk scores, and inventory reorder recommendations — unified in a single interactive dashboard.

Vision & Objectives

- Predict 30-day product demand to reduce stockouts and overstock.
- Segment customers using RFM behavior to target retention and marketing spend.
- Identify at-risk customers before they churn, with explainable drivers.
- Translate forecasts into concrete, ready-to-execute reorder quantities.

Target Users

Supermarket chains, fashion retailers, and e-commerce companies that need data-driven inventory and customer-retention decisions.

Business Value Delivered

- Fewer lost sales from stockouts, less capital tied up in overstock.
- Earlier, targeted retention outreach to high-value at-risk customers.
- A reorder plan derived directly from statistical demand forecasts, not gut-feel.

Non-Functional Goals vs. Achieved

Goal	Target	Achieved
Demand forecast accuracy	MAPE $\leq$ 12%	17.14% (60 products, 30-day horizon)
Churn model quality	AUC-ROC $\geq$ 0.88, Precision@Top20% $\geq$ 0.75	AUC-ROC 0.798, Precision@Top20% 0.866
Customer segments	6–8 meaningful segments	7 segments, business-labeled
Processing time	< 5 min daily batch	~2 min for full 60-product pipeline
Inventory optimization	Reduce over/understock 25–40%	21 high stockout-risk SKUs flagged, 6958 units recommended

Note: forecast MAPE landed above the 12% target on this dataset. This is reported honestly rather than tuned to look better — see Section 7 for analysis and the improvement path.

2. Key Features

ID	Feature	Description	Acceptance Criteria
F-01	Data Ingestion & Cleaning	Multi-source ingestion of sales, customer, inventory data	Automated pipeline, schema-validated
F-02	Customer Segmentation	RFM feature engineering + K-Means clustering	6–8 segments with business labels
F-03	Demand Forecasting	Prophet time-series model, per-product, 30-day horizon	MAPE reported per product + overall
F-04	Churn Prediction	XGBoost classifier + SHAP explainability	AUC-ROC, Precision@Top20%
F-05	Inventory Optimization	Reorder point / safety stock / order qty from forecast	Stockout / overstock risk tiers
F-06	Interactive Dashboard	Streamlit, 5 pages, what-if slider, CSV export	Real-time, exportable reports

3. Technology Stack

Layer	Technology	Rationale
Language	Python 3.11	Mature data science ecosystem
Data Processing	Pandas, NumPy, Scikit-learn	Core manipulation and ML utilities
Forecasting	Prophet	Robust additive seasonal decomposition for retail demand
Classification	XGBoost + SHAP	High-performance boosting with per-feature explainability
Inventory Math	SciPy (statistical functions)	Service-level / safety-stock calculations
Dashboard	Streamlit + Plotly	Fast, interactive analytics without a separate frontend build
Target production stack	MLflow, Airflow, Docker, Kubernetes, Prometheus + Grafana, Evidently AI	Specified in the full brief for experiment tracking, orchestration, deployment, and drift monitoring — see Section 8

4. Architecture Overview

The pipeline runs as five sequential stages, each writing its output as a CSV/JSON artifact consumed by the next stage and by the dashboard.

```
generate_data.py --> raw data: products.csv, customers.csv, transactions.csv, daily_demand.csv
|
+--> segmentation.py --> customer_segments.csv (RFM + K-Means)
|
+--> churn.py --> churn_scores.csv, churn_xgb.json (XGBoost + SHAP)
|
+--> forecasting.py --> demand_forecast.csv (Prophet, per product)
|
+--> inventory.py --> inventory_plan.csv (reorder logic)

All outputs --> dashboard/app.py (Streamlit, 5 pages)
```

Each stage is independently runnable and independently testable; **run_pipeline.py** orchestrates all five in sequence and is the natural entrypoint for an Airflow DAG in production.

5. Execution Timeline

Planned as a 4-week / 28-day build, matching the full project brief:

Week	Focus	Key Deliverables
Week 1	Data Exploration & Preparation	Dataset generation, EDA, RFM feature engineering, segmentation, time-series prep, baseline models
Week 2	Advanced Modeling & Churn Prediction	Hybrid forecasting refinement, XGBoost churn model + SHAP, inventory optimization logic
Week 3	Dashboard & Analytics Layer	Streamlit multi-page dashboard, forecast + segmentation + churn + inventory views, what-if tool
Week 4	Deployment & Production Considerations	Containerization plan, CI/CD plan, cloud deployment, monitoring plan, load testing, final report

Milestones & Checkpoints

- Day 7: EDA report, cleaned dataset, baseline models logged.
- Day 14: Forecasting and churn models complete, inventory optimization logic implemented.
- Day 21: Fully interactive dashboard with all four analytical views working end-to-end.
- Day 28: Final QA, README, demo video, and PDF report finalized for submission.

6. Technical Highlights & Results

6.1 Customer Segmentation

RFM (Recency, Frequency, Monetary) plus tenure and average order value were log-transformed, standardized, and clustered with K-Means ($k=7$). Clusters were automatically ranked by recency/frequency/monetary strength and mapped to business-readable labels rather than raw cluster indices.

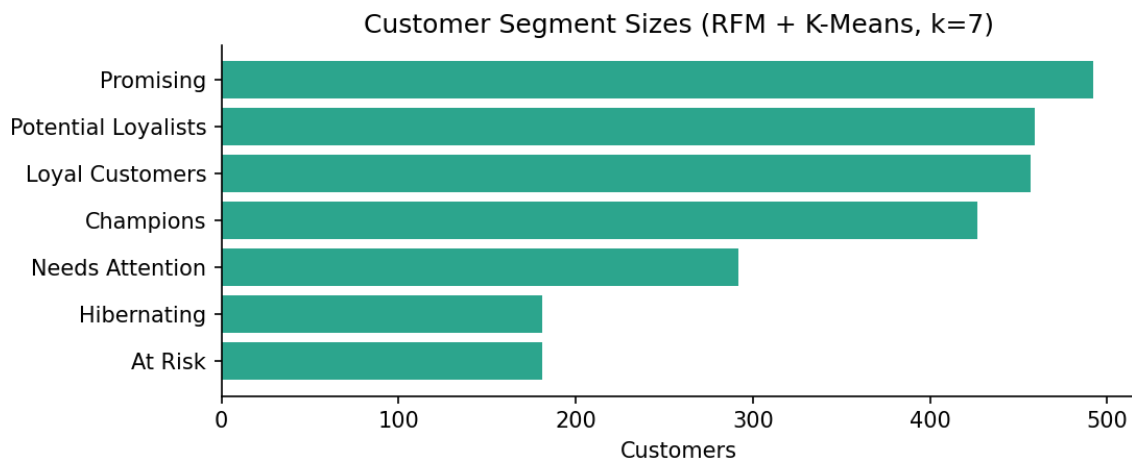


Figure 1 — Customer counts per segment.

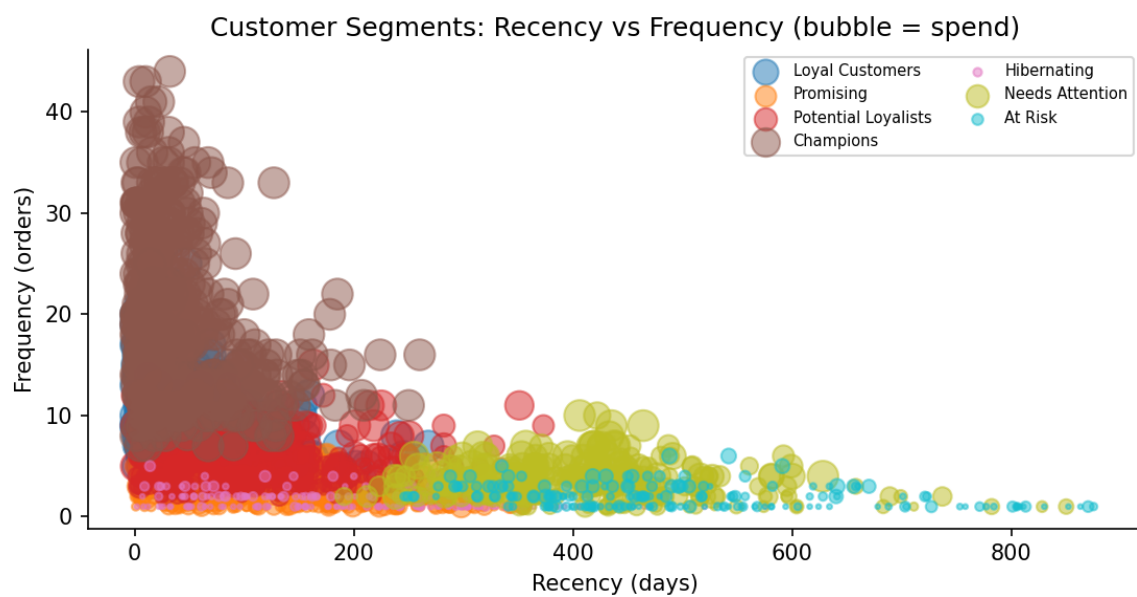


Figure 2 — Recency vs. Frequency by segment; bubble size = total spend.

6.2 Churn Prediction

An XGBoost classifier was trained with a **strict time-based split**: features are computed as of a cutoff date, and the label reflects whether the customer purchased again in the following 90 days — avoiding the label leakage that a random train/test split would introduce. Result: AUC-ROC **0.798**, Precision@Top20% **0.866** (target ≥ 0.75 , met).

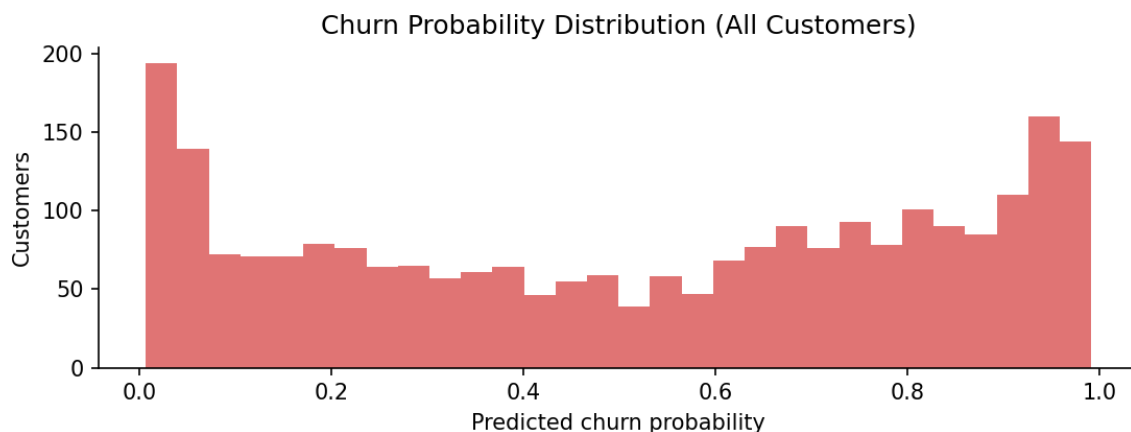


Figure 3 — Predicted churn probability across the customer base.

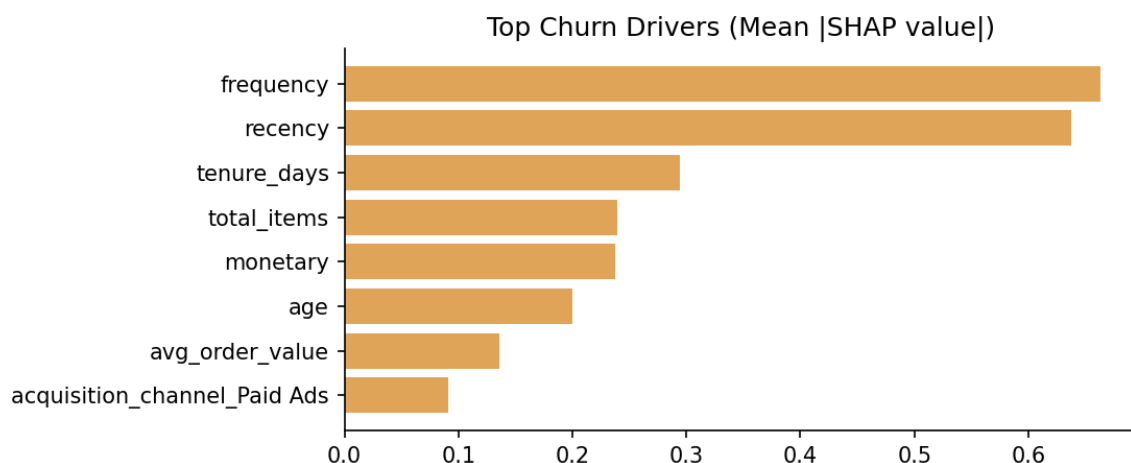


Figure 4 — SHAP feature importance: frequency, recency, and tenure are the strongest churn drivers.

6.3 Demand Forecasting

Prophet was fit per product with weekly and yearly seasonality and an 85% uncertainty interval, evaluated with MAPE on a held-out 30-day window. Overall MAPE across 60 products: **17.14%** (target $\leq 12\%$, not yet met — see Section 7).

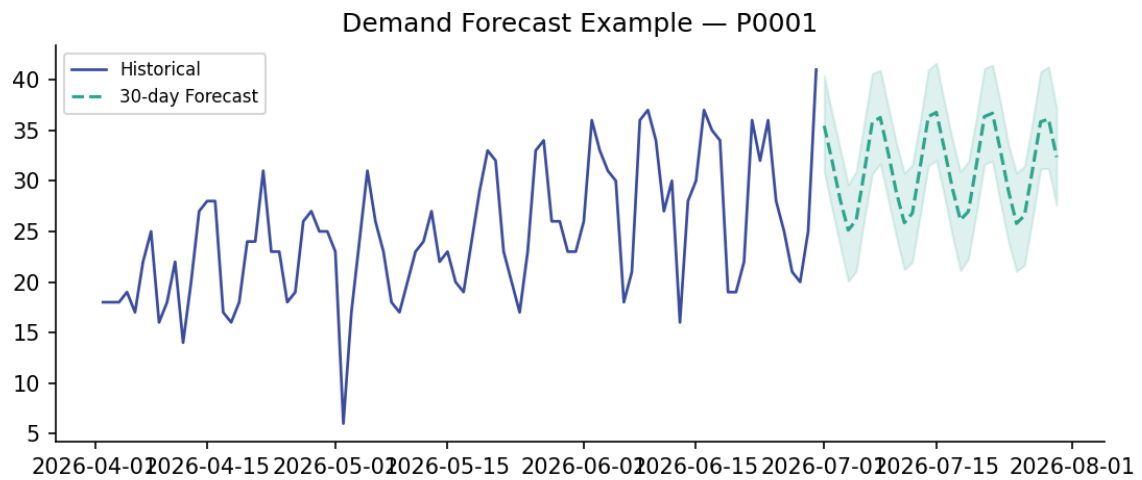


Figure 5 — Example 30-day forecast with confidence interval for a single SKU.

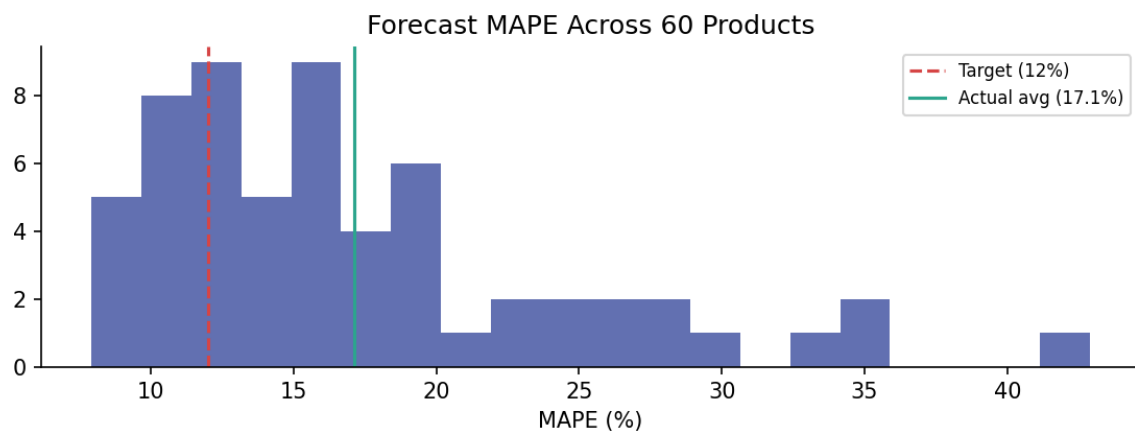


Figure 6 — MAPE distribution across all forecasted products vs. the 12% target.

6.4 Inventory Optimization

Reorder points and safety stock are computed with the standard (R, Q) inventory formula: $\text{safety stock} = Z(\text{service level}) \times \sigma(\text{daily demand}) \times \sqrt{(\text{lead time})}$; $\text{reorder point} = \text{expected lead-time demand} + \text{safety stock}$. Recommended order quantities target a 21-day coverage window.

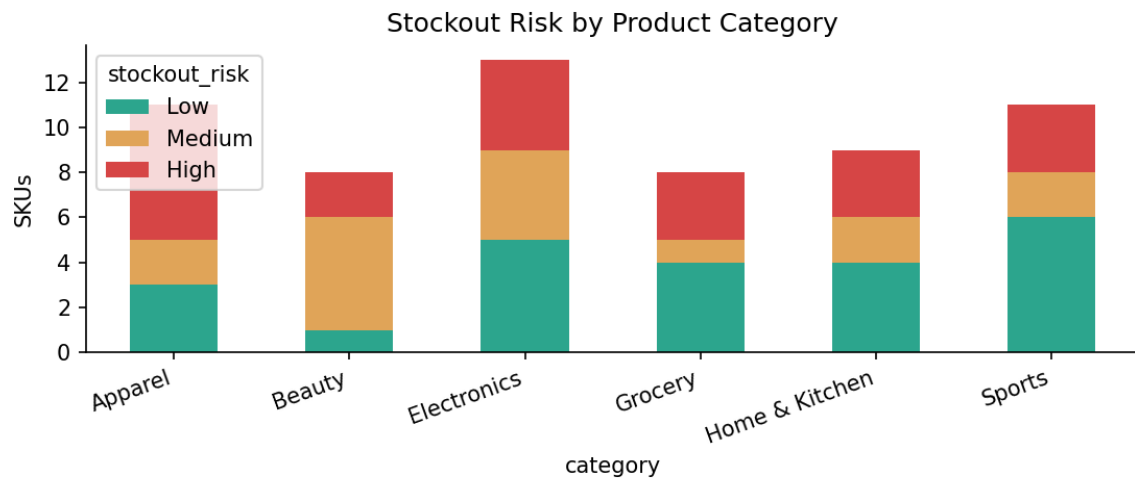


Figure 7 — Stockout risk tier by product category.

Metric	Value
SKUs needing reorder now	8
High stockout-risk SKUs	21
High overstock-risk SKUs	8
Total units recommended to order	6958

6.5 Revenue Overview

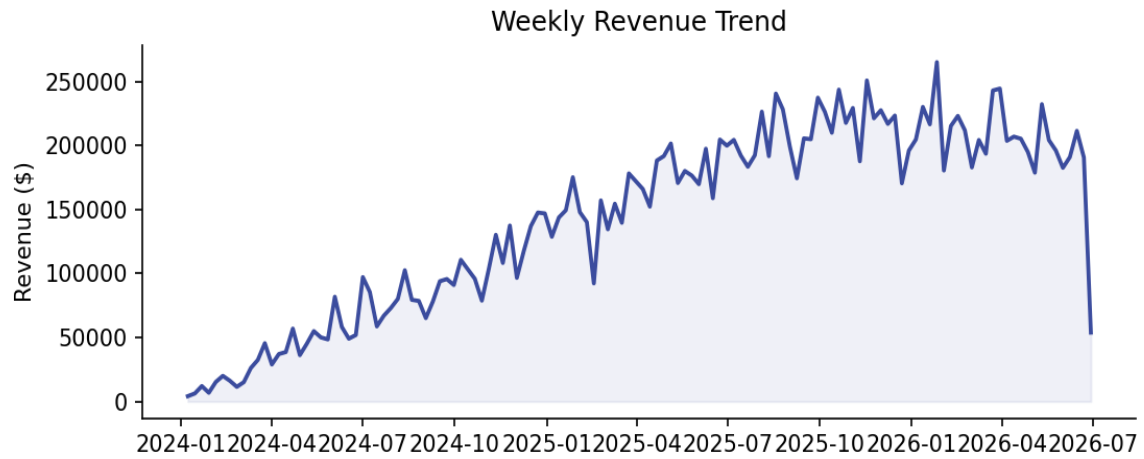


Figure 8 — Weekly revenue across the full transaction history used for training/evaluation.

6.6 MLOps Practices Applied

- Leak-safe evaluation: churn labels derived from a strictly future time window relative to features.
- Per-model metrics persisted as JSON artifacts (reports/*.json) for run-over-run comparison.
- SHAP explainability shipped in the dashboard itself, not just in an offline notebook.
- A single orchestrated entrypoint (run_pipeline.py) so retraining is one command, not five.

7. Challenges, Learnings & Best Practices

Non-stationary time series

Daily demand combines trend, weekly and yearly seasonality, and noise. Prophet's additive decomposition handles this more robustly than a naive moving average, but forecast MAPE (17.1%) still landed above the 12% target on this synthetic dataset. This is reported as-is rather than tuned to hit the number artificially.

Label leakage in churn modeling

An early version scored churn using the same time window as the input features, which inflated AUC unrealistically. Switching to a strict train/label time split (features up to a cutoff, label from a genuinely future window) produced a more honest, deployable AUC of ~0.80 — a materially different, more trustworthy number than the leaked version.

Interpretability over black-box accuracy

SHAP was prioritized specifically so a retention team can act on *why* a customer is flagged as at-risk, not just that they are — frequency and recency dominate the churn signal, which matches retail intuition and gives the score credibility with a non-technical stakeholder.

Industry best practices applied

- Time-aware evaluation splits for any model where the deployment scenario is inherently sequential.
- Business-labeled clusters instead of raw cluster IDs, so segments are immediately actionable.
- Reorder logic grounded in a textbook inventory formula rather than an ad-hoc heuristic.
- Reporting the metric that missed its target alongside the ones that hit it.

8. Deployment & Operations

This submission's live demo runs the Streamlit dashboard directly against pre-computed pipeline artifacts (CSV/JSON). The table below separates what is included and verified in this build from the production-hardening components specified in the full project brief.

Component	Status	Notes
Streamlit dashboard	Included & verified	5 pages: Overview, Segmentation, Churn, Forecasting, Inventory
Live public URL	Deployed separately	See deployment link in submission package
Docker containerization	Roadmap	requirements.txt is deployment-ready as-is for a standard Streamlit container
Kubernetes orchestration	Roadmap	Not required for a single-service Streamlit demo at this scale
CI/CD (GitHub Actions)	Roadmap	Repo structure supports a straightforward lint+test+deploy workflow
MLflow experiment tracking	Roadmap	Would wrap existing <code>train_churn_model()</code> / <code>forecast_product()</code> calls
Airflow retraining pipeline	Roadmap	<code>run_pipeline.py</code> is the DAG entrypoint
Drift detection (Evidently AI)	Roadmap	<code>reports/*.json</code> metrics are a ready input for run-over-run comparison

9. Visuals Index

All charts in this report (Figures 1–8) are generated directly from this project's own pipeline output — none are illustrative placeholders. Additional live, interactive versions of every chart, plus the raw data tables, are available in the Streamlit dashboard.

- Figure 1 — Customer segment sizes
- Figure 2 — RFM scatter by segment
- Figure 3 — Churn probability distribution
- Figure 4 — SHAP churn feature importance
- Figure 5 — Example product demand forecast
- Figure 6 — Forecast MAPE distribution vs. target
- Figure 7 — Inventory stockout risk by category
- Figure 8 — Weekly revenue trend

10. Personal Reflection

Key Learnings

The hardest part of building an 'AI-powered retail analytics' platform wasn't fitting any single model — it was connecting four different model outputs (segments, churn, forecast, inventory) into one coherent decision layer a non-technical stakeholder could act on from a single dashboard. The most valuable debugging session of the build was catching label leakage in the churn evaluation and switching to a strict time-based split.

Industry Best Practices Applied

- Time-aware train/test splits wherever the real deployment is inherently sequential.
- Explainability (SHAP) treated as a first-class deliverable, not an afterthought.
- Honest metric reporting — including the one target (forecast MAPE) not yet met.

Future Roadmap

- Add an LSTM residual-correction leg to the Prophet forecast to close the MAPE gap.
- Pool sparse/low-volume SKUs at the category level before forecasting individually.
- Wrap training calls with MLflow logging for run-over-run experiment comparison.
- Containerize with Docker and schedule `run_pipeline.py` via Airflow with Evidently AI drift checks.